
Integration Manual

for MPC560XB FLS Driver

Document Number: IM13FLSASR4.0 Rev0003R1.0.1
Rev 3.0





Contents

Section number	Title	Page
----------------	-------	------

Chapter 1 Revision History

Chapter 2 Introduction

2.1	Supported Derivatives.....	7
2.2	Overview.....	8
2.3	About this Manual.....	8
2.4	Acronyms and Definitions.....	8
2.5	Reference List.....	9

Chapter 3 Building the Driver

3.1	Build Options.....	11
3.1.1	CW Compiler/Linker/Assembler Options.....	11
3.1.2	DIAB Compiler/Linker/Assembler Options.....	13
3.1.3	GHS Compiler/Linker/Assembler Options.....	15
3.1.4	HT Compiler/Linker/Assembler Options.....	16
3.2	Files required for Compilation.....	17
3.3	Setting up the Plug-ins.....	19

Chapter 4 Function calls to module

4.1	Function Calls during Start-up.....	21
4.2	Function Calls during Shutdown.....	21
4.3	Function Calls during Wake-up.....	21
4.4	Implementing an Exception Handler in case of non correctable ECC error.....	22

Chapter 5 Module requirements

5.1	Exclusive areas to be defined in BSW scheduler.....	25
5.2	Peripheral Hardware Requirements.....	26

Section number	Title	Page
5.3	ISR to configure within OS – dependencies.....	27
5.4	ISR Macro.....	27
5.5	Other AUTOSAR modules - dependencies.....	27

Chapter 6 Main API Requirements

6.1	Main functions calls within Rte module.....	29
6.2	API Requirements.....	29
6.3	Calls to Notification Functions, Callbacks, Callouts.....	29
6.4	Tips for FLS integration.....	29

Chapter 7 Memory Allocation

7.1	Sections to be defined in MemMap.h.....	33
7.2	Linker command file.....	34

Chapter 8 Configuration parameters considerations

8.1	Configuration Parameters.....	37
-----	-------------------------------	----

Chapter 9 Integration Steps

Chapter 10 ISR Reference

10.1	Software specification.....	41
10.1.1	Define Reference.....	41
10.1.2	Macros Reference.....	41
10.1.3	Enum Reference.....	41
10.1.4	Function Reference.....	41
10.1.5	Structs Reference.....	41
10.1.6	Types Reference.....	42
10.1.7	Variables Reference.....	42

Chapter 1

Revision History

Table 1-1. Revision History

Revision	Date	Author	Description
1.0	16-Feb-2012	Cosimo Stornaiuolo	Initial Release for MPC560XB
2.0	14-May-2013	Khanindra Deka	Updated for the MPC560XB RTM 1.0.0
3.0	06-Jun-2014	Guillaume Herry	Updated for RTM 1.0.1 Release - Exclusive areas section updated - Note in ECC error management removed - Exc_types.h file removed



Chapter 2

Introduction

This integration manual describes the integration requirements for Fls Driver for MPC560XB microcontrollers.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of Freescale Semiconductor .

Table 2-1. MPC560XB Derivatives

Freescale Semiconductor	mpc5601dxlh_lqfp64 mpc5601dxll_lqfp100 mpc5602dxlh_lqfp64 mpc5602dxll_lqfp100 mpc5602bxlh_lqfp64 mpc5602bxll_lqfp100 mpc5602bxlq_lqfp144 mpc5602cxlh_lqfp64 mpc5602cxll_lqfp100 mpc5603bxlh_lqfp64 mpc5603bxll_lqfp100 mpc5603bxlq_lqfp144 mpc5603cxlh_lqfp64 mpc5603cxll_lqfp100 mpc5604bxlh_lqfp64 mpc5604bxll_lqfp100 mpc5604bxlq_lqfp144 mpc5604bxmg_mapbga208 mpc5604cxlh_lqfp64 mpc5604cxll_lqfp100 mpc5605bxll_lqfp100 mpc5605bxlq_lqfp144 mpc5605bxlu_lqfp176 mpc5606bkll_lqfp100 mpc5606bxlq_lqfp144 mpc5606bxlu_lqfp176 mpc5607bxlu_lqfp176 mpc5607bxmg_mapbga208
-------------------------	--

All of the above microcontroller devices are collectively named as MPC560XB .

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About this Manual

This Technical Reference employs the following typographical conventions:

Boldface type: Bold is used for important terms, notes and warnings.

Italic font: Italic typeface is used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

2.4 Acronyms and Definitions

Table 2-2. Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
VLE	Variable Length Encoding
N/A	Not Applicable
MCU	Micro Controller Unit
ECU	Electronic Control Unit
EEPROM	Electrically Erasable Programmable Read-Only Memory
FEE	Flash EEPROM Emulation
FLS	Flash
XML	Extensible Markup Language

2.5 Reference List

Table 2-3. Reference List

#	Title	Version
1	AUTOSAR 4.0 Rev0003Fls Driver Software Specification Document.	V4.0 Rev003
2	MPC5607B Reference Manual	Rev. 7.2, May 2012
3	MPC5604B Reference Manual	Rev. 8.2, September 2013
4	5606B Reference Manual	Rev. 2, May 2014
5	5602D Reference Manual	Rev. 4.2, Dec 2013
6	Bolero 1.5M cut 2.2 Errata Sheet (MPC5607B_1M03Y)	Rev. 12Dec2013
7	Bolero 512k cut 2.3 Errata Sheet (MPC5604B_2M27V)	Rev. 26 FEB 2014
8	Bolero 256k cut 1.1 Errata Sheet (MPC5602D_1M18Y)	Rev. , 12 DEC 2013

Chapter 3

Building the Driver

This section describes the source files and various compilers, linker options used for building the Autosar Fls driver for Freescale Semiconductor MPC560XB . It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

The Fls driver files are compiled using

- Green Hills MULTI v6.1.4
- Windriver DIAB Rel DIAB_5_9_2_0-FCS_20120725_234511
- Freescale CodeWarrior for MPC5xxx v2.10 - compiler Version 4.3 build 224
- HighTec v4.6.4.0, Compiler gcc v4.6.3

The compiler, linker flags used for building the driver are explained below:

Note

The TS_T2D13M10I1R0 plugin name is composed as follow:

TS_T = Target_Id

D = Derivative_Id

M = SW_Version_Major

I = SW_Version_Minor

R = Revision

(i.e. Target_Id = 2 identifies PowerPC architecture and
Derivative_Id = 13 identifies the MPC560XB)

3.1.1 CW Compiler/Linker/Assembler Options

Table 3-1. Compiler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-lang c	Expects source code to conform to the language specified by the ISO/IEC 9899-1990 ("C90") standard
-opt all	This option is selected all optimization (the same as -opt speed,level=4,intrinsics,noframe)
-common off	Disables moving uninitialized data into a common section
-sdatathreshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small data. (The linker stores small data items in the Small Data address space. The compiler can generate faster code to access this data.)
-sdata2threshold 0	Specifies the threshold size (in bytes) for an item considered by the linker to be small constant data. (The linker stores small constant data items in the Small Constant Data address space.)
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-use_lmw_stmw on	Enables the use of multiple load and store instructions for function prologues and epilogues
-ppc_asm_to_vle	Converts regular Power Architecture assembler mnemonics to equivalent VLE (Variable Length Encoded) assembler mnemonics in the inline assembler
-Cpp_exceptions off	When on, generates executable code for C++ exceptions. When off, generates smaller, faster executable code
-func_align 4	Specifies alignment of functions in executable code
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format
-w on	Turns on most warning messages
-r	Compiler should expect function prototypes
-w undefmacro	Issues warning messages on the use of undefined macros in #if and #elif conditionals
-char unsigned	Controls the default sign of the char data type: char data items are unsigned
-nosyspath	Performs a search of both the user and system paths, treating #include statements of the form #include xyz the same as the form #include "xyz"
-fp none	No floating point code generation
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-2. Assembler Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch

Table continues on the next page...

Table 3-2. Assembler Options (continued)

Option	Description
-vle	Tells the compiler and linker to generate and lay out Variable Length Encoded (VLE) instructions, available on Zen variants of Power Architecture processors
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-gdwarf-2	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format). The linker ignores debugging information that is not in the Dwarf 1, Dwarf 2 format.
-DMWERKS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the CW preprocessor symbol.

Table 3-3. Linker Options

Option	Description
-proc Zen	Generates and links object code for Zen processor. The compiler uses unsigned as the default parameter for the -char switch
-code_merging all	Removes duplicated functions to reduce object code size
-far_near_addressing	Simplifies address computations to reduce object code size and improve performance
-vle_enhance_merging	Removes duplicated functions that are called by functions that use VLE instructions to reduce object code size
-listdwarf	DWARF debugging information in the linker's map file
-sym dwarf-2,full	Generate DWARF-2-conforming debugging information (Debug With Arbitrary Record Format)
-char unsigned	Controls the default sign of the char data type: char data items are unsigned.

3.1.2 DIAB Compiler/Linker/Assembler Options

Table 3-4. Compiler Options

Option	Description
-Xdialect-ansi	Follow the ANSI C standard with some additions
-XO	Enables extra optimizations to produce highly optimized code
-g	Generate symbolic debugger information and disable some optimizations.
-Xsize-opt	Optimize for size rather than speed when there is a choice
-Xsmall-data=0	Set Size Limit for "small data" Variables to zero.
-Xaddr-sconst=0x11	Specify addressing for constant static and global variables with size less than or equal to -Xsmall-const to far-absolute.
-Xaddr-sdata=0x11	Specify addressing for non-constant static and global variables with size less than or equal to -Xsmall-data in size to far-absolute.
-Xno-common	Disable use of the "COMMON" feature so that the compiler or assembler will allocate each uninitialized public variable in the .bss section for the module defining it, and the linker will require exactly one definition of each public variable
-Xnested-interrupts	Allow nested interrupts

Table continues on the next page...

Table 3-4. Compiler Options (continued)

Option	Description
-Xdebug-dwarf2	Generate symbolic debug information in dwarf2 format
-Xdebug-local-all	Force generation of type information for all local variables
-Xdebug-local-cie	Create common information entry per module
-Xdebug-struct-all	Force generation of type information for all typedefs, struct, union and class types
-Xforce-declarations	Generates warnings if a function is used without a previous declaration
-ee1481	Generate an error when the function was used before it has been declared
-Xmacro-undefined-warn	Generates a warning when an undefined macro name occurs in a #if preprocessor directive
-W:as;,-l	Pass the option "-l" (lower case letter L) to the assembler to get an assembler listing file
-Wa,-Xisa-vle	Instruct the assembler to expect and assemble VLE (Variable Length Encoding) instructions rather than BookE instructions.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DDIAB	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the DIAB preprocessor symbol.
-Xforce-prototypes	-Generate warnings if a function is used without a previous prototype declaration.
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries.

Table 3-5. Assembler Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-g	Dump the symbols in the global symbol table in each archive file.
-Xisa-vle	Expect and assemble VLE (Variable Length Encoding) instructions rather than Book E instructions. The default code section is named .text_vle instead of .text, and the default code section fill "character" is set to 0x44444444 instead of 0. The .text_vle code section will have ELF section header flags marking it as VLE code, not Book E code.

Table 3-6. Linker Options

Option	Description
-tPPCE200Z0VEN:simple	Sets target processor to PPCE200Z0, generates ELF using EABI conventions, No floating point support (minimizes the required runtime), selects simple environment settings for Startup Module and Libraries
-Xelf	Generates ELF object format for output file
-m6	Generates a detailed link map and cross reference table
-lc	Specifies to linker to search for libc.a
-Xlibc-old	Enables usage of legacy (pre-release 5.6) libraries

3.1.3 GHS Compiler/Linker/Assembler Options

Table 3-7. Compiler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-ansi	Specifies ANSI C with extensions. This mode extends the ANSI X3.159-1989 standard with certain useful and compatible constructs.
-noSPE	Disables the use of SPE and vector floating point instructions by the compiler.
-Ospace	Optimize for size
-sda=0	Enables the Small Data Area optimization with a threshold of 0.
-vle	Enables VLE code generation
-dual_debug	Enables the generation of DWARF, COFF, or BSD debugging information in the object file
-G	Generates source level debugging information and allows procedure call from debugger's command line.
--no_exceptions	Disables support for exception handling
-Wundef	Generates warnings for undefined symbols in preprocessor expressions
-Wimplicit-int	Issues a warning if the return type of a function is not declared before it is called
-Wshadow	Issues a warning if the declaration of a local variable shadows the declaration of a variable of the same name declared at the global scope, or at an outer scope
-Wtrigraphs	Issues a warning for any use of trigraphs
--prototype_errors	Generates errors when functions referenced or called have no prototype
--incorrect_pragma_warnings	Valid #pragma directives with wrong syntax are treated as warnings
-noslashcomment	C++ like comments will generate a compilation error
-preprocess_assembly_files	Preprocesses assembly files
--short_enum	Store enumerations in the smallest possible type
--no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup.
-c	Produces an object file (called input-file.o) for each source file.
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.
-DDISABLE_MCAL_INTERMODULE_ASR_CHECK	-D defines a preprocessor symbol to disable the inter-module version check for AR_RELEASE versions. DISABLE_MCAL_INTERMODULE_ASR_CHECK: By default in the package, drivers are compiled to perform the inter-module version check as per Autosar BSW004. When the inter-module version check needs to be disabled then the DISABLE_MCAL_INTERMODULE_ASR_CHECK global define must be added to the list of compiler options.
-DGHS	-D defines a preprocessor symbol and optionally can set it to a value. This one defines the GHS preprocessor symbol.

Table 3-8. Assembler Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb

Table 3-9. Linker Options

Option	Description
-cpu=ppc560xb	Selects target processor: ppc560xb
-nostartfiles	Do not use Start files.
-vle	Enables VLE code generation
-linker_warnings	Display linker warnings

3.1.4 HT Compiler/Linker/Assembler Options

Table 3-10. Compiler Options

Option	Description
-Os	Optimize for size. '-Os' enables all '-O2' optimizations that do not typically increase code size. It also performs further optimizations designed to reduce code size.
-msdata=eabi	On System V.4 and embedded PowerPC systems, put small initialized const global and static data in the '.sdata2' section, which is pointed to by register r2. Put small initialized non-const global and static data in the '.sdata' section, which is pointed to by register r13. Put small uninitialized global and static data in the '.sbss' section, which is adjacent to the '.sdata' section.
-G1000	On embedded PowerPC systems, put global and static items less than or equal to num bytes into the small data or bss sections instead of the normal data or bss section.
-fshort-double	Use the same size for double as for float.
-mno-spe	This option used to disable spe extension (Like -noSPE on ghs compiler).
-mregnames	On System V.4 and embedded PowerPC systems do (do not) emit register names in the assembly language output using symbolic forms
-g3	Request debugging information of Level 3. Includes extra information, such as all the macro definitions present in the program.
-Wall	This enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros.
-mvle	Enable support of VLE
--mcpu=z425n3	Support of e200 Z425n3 core
-DAUTOSAR_OS_NOT_USED	-D defines a preprocessor symbol and optionally can set it to a value. AUTOSAR_OS_NOT_USED: By default in the package, the drivers are compiled to be used without Autosar OS. If the drivers are used with Autosar OS, the compiler option '-DAUTOSAR_OS_NOT_USED' must be removed from project options
-DUSE_SW_VECTOR_MODE	-D defines a preprocessor symbol and optionally can set it to a value. USE_SW_VECTOR_MODE: By default in the package, drivers are compiled to be used with interrupt controller configured to be in hardware vector mode. In case of AUTOSAR_OS_NOT_USED, the compiler option "-DUSE_SW_VECTOR_MODE" must be

Table 3-10. Compiler Options

Option	Description
	added to the list of compiler options to be used with interrupt controller configured to be in software vector mode.

Table 3-11. Assembler Options

Option	Description
-c	Compile or assemble the source files, but do not link. The linking stage simply is not done. The ultimate output is in the form of an object file for each source file.
-Wa,--gdwarf2	Produce debugging information in DWARF format.
-msdata=eabi	Put small initialized const global and static data in the .sdata2 section, which is pointed to by register r2. Put small initialized non-const global and static data in the .sdata section, which is pointed to by register r13. Put small uninitialized global and static data in the .sbss section, which is adjacent to the .sdata section.
-mregnames	Do emit register names in the assembly language output using symbolic forms.
-mno-spe	This option used to disable spe extension (Like --noSPE on ghs compiler).
-mcpu=z425n3	Support of e200 z425n3 core

Table 3-12. Linker Options

Option	Description
-N	Set the text and data sections to be readable and writable. Also, do not page-align the data segment. If the output format supports Unix style magic numbers, mark the output as OMAGIC.
--no-undefined	Normally when a symbol has an undefined version, the linker will ignore it. This option disallows symbols with undefined version and a fatal error will be issued instead.
--warn-common	Normally ppc-vle-ld will give a warning if it finds an incompatible library during a library search. This option silences the warning.
--error-unresolved-symbols	This restores the linker's default behaviour of generating errors when it is reporting unresolved symbols.
-nocrt0	library crt0 from compiler library not used, it uses custom crt0 file
-nostartfiles	no start file from compiler library is used
-Wl	If the linker is being invoked indirectly, via a compiler driver (e.g. 'gcc') then all the linker command line options should be prefixed by '-Wl,'Used in combination with -no-warn-flags, --no-undefined, --warn-common and --error-unresolved-symbols
--no-warn-flags	Used in combination with -Wl disable linker warnings. See previous
-T	The -T option specifies the name of your own script ie sprig.link. The -c is a synonym of the -T option

3.2 Files required for Compilation

This section describes the include files required to compile, assemble (if assembler code) and link the Fls driver for MPC560XB microcontrollers.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR major and minor versions can be compiled.

Fls Files

- ..\Fls_TS_T2D13M10I1R0\src\Fls.c
- ..\Fls_TS_T2D13M10I1R0\src\Fls_Ac.c
- ..\Fls_TS_T2D13M10I1R0\include\Fls.h
- ..\Fls_TS_T2D13M10I1R0\include\Fls_Api.h
- ..\Fls_TS_T2D13M10I1R0\include\Fls_InternalTypes.h
- ..\Fls_TS_T2D13M10I1R0\include\Fls_Types.h
- ..\Fls_TS_T2D13M10I1R0\include\Fls_Version.h
- ..\Fls_TS_T2D13M10I1R0\include\Reg_eSys_FLASHC.h
- Fls_PBcfg.c - this file should be generated by the user using a configuration/generation tool
- Fls_Cfg.h - this file should be generated by the user using a configuration/generation tool
- Fls_Cfg.c - this file should be generated by the user using a configuration/generation tool
- Note: Fls_Ac.c that implements Erase/Write access codes are implemented in C language.

Other includes files:

Files from MemIf folder:

- ..\MemIf_TS_T2D13M10I1R0\include\MemIf_Types.h

Files from Base common folder

- ..\Base_TS_T2D13M10I1R0\include\Compiler.h
- ..\Base_TS_T2D13M10I1R0\include\Compiler_Cfg.h
- ..\Base_TS_T2D13M10I1R0\include\ComStack_Types.h
- ..\Base_TS_T2D13M10I1R0\include\MemMap.h
- ..\Base_TS_T2D13M10I1R0\include\Mcal.h
- ..\Base_TS_T2D13M10I1R0\include\Platform_Types.h
- ..\Base_TS_T2D13M10I1R0\include\Std_Types.h
- ..\Base_TS_T2D13M10I1R0\include\Reg_eSys.h
- ..\Base_TS_T2D13M10I1R0\include\Soc_Ips.h
- ..\Base_TS_T2D13M10I1R0\include\Reg_Macros.h

Files from Dem folder:

- ..\Dem_TS_T2D13M10I1R0\include\Dem.h

Files from Det folder:

- ..\Det_TS_T2D13M10I1R0\include\Det.h

Files from RTE folder:

- ..\Rte_TS_T2D13M10I1R0\include\SchM_Fls.h

3.3 Setting up the Plug-ins

The Fls driver was designed to be configured by using the EB Tresos Studio (version Elektrobit Tresos Release 14.2.1 (b140128-1223) or later.)

- VSMD (Vendor Specific Module Definition) file in EB tresos Studio XDM format: ..\Fls_TS_T2D13M10I1R0\config\Fls.xdm
- VSMD (Vendor Specific Module Definition) file in AUTOSAR compliant EPD format: ..\Fls_TS_T2D13M10I1R0\autosar\ (one EPD file for each supported sub-derivative)
- Code Generation Templates for Post-Build time configuration parameters:
 - ..\Fls_TS_T2D13M10I1R0\generate\include\Fls_Cfg.h
 - ..\Fls_TS_T2D13M10I1R0\generate\src\Fls_PBcfg.c
- Code Generation Templates for Pre-Compile time configuration parameters:
 - ..\Fls_TS_T2D13M10I1R0\generate\include\Fls_Cfg.h
 - ..\Fls_TS_T2D13M10I1R0\generate\src\Fls_Cfg.c

Steps to generate the configuration:

1. Copy the module folders Fls_TS_T2D13M10I1R0 , Base_TS_T2D13M10I1R0 and Resource_TS_T2D13M10I1R0 into the Tresos plugins folder.
2. Set the desired Tresos Output location folder for the generated sources and header files.
3. Use the EB tresos Studio GUI to modify ECU configuration parameters values.
4. Generate the configuration files.

Chapter 4

Function calls to module

None.

4.1 Function Calls during Start-up

Fls shall be initialized during STARTUP phase of EcuM initialization. The API to be called for this is Fls_Init().

The MCU module should be initialized before the Fls is initialized.

If the FLS driver is used in user mode, be sure that the Flash memory controller registers are accessible and that accessed Flash memory partition is not protected. For more information please refer to the "Memory Protection Unit" and "Register Protection" chapters in the device reference manual.

The Flash memory physical sectors that are going to be modified by Fls driver (i.e. erase and write operations) have to be unlocked for a successful operation. It is also handled by Fls driver, but it should be configured using Fls_PhysicalSectorUnlock parameter for all configured blocks.

Note: if the unlock is not handled by Fls driver must be setup on an application level. Example of unlock code can be found in Fls driver UM document.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

4.4 Implementing an Exception Handler in case of non correctable ECC error

When reading a FLASH location with a non correctable ECC error an IVOR exception is thrown.

On Z0, Z3 and Z6 core based platforms, a different IVOR is thrown according to the value of the bits ME and EE in the MSR (Machine Check Register) as show in the table below:

Table 4-1. IVOR selection on Z0, Z3 and Z6 core based platform

MSR[ME]	MSR[EE]	data/instruction	IVOR	Function to call
0	0	X	N/A	No IVOR thrown: Machine Checkstop state is entered (TO BE AVOIDED)
1	1	flash data read	IVOR2 (DSI)	Fls_DsiHandler
1	0	flash data read	IVOR1 (MCI)	Fls_MciHandler

On Z4 and Z7 core based platform, IVOR1 is thrown independently from MSR bits state as show in the table below.

Table 4-2. IVOR selection on Z4 and Z7 core based platform

MSR[ME]	MSR[EE]	data/instruction	IVOR	Function to call
0	0	X	N/A	Machine check stop not implemented (can cause infinite loop, TO BE AVOIDED)
1	0, 1	flash data read	IVOR1	Fls_DsiHandler

The Flash driver provide two API which can be called inside the user ECC error recovery handler (IVOR handler driver functions):

- **Fls_CompHandlerReturnType Fls_DsiHandler (Fls_ExceptionDetailsType *);**
- **Fls_CompHandlerReturnType Fls_MciHandler (Fls_ExceptionDetailsType *);**

For Z0, Z3 and Z6 both functions should be used as explained on above tables, while for Z4 and Z7 core only **Fls_DsiHandler()** must be used.

Those API's are available only if the configuration parameter **FlsDsiHandlerApi = true;**

The **Fls_ExceptionDetailsType** data structure contains some information about the details of Exception and in particular:

- a pointer to the statement that generated the ECC (**Fls_InstructionAddressType**);
- an address of the data that caused the error in ECC (**Fls_DataAddressType**);
- details on the type of exception (**uint32**).

IVOR Handler Driver functions examine the job executed by the driver and the data contained in the structure **Fls_ExceptionDetailsType** in particular:

- Check whether there is pending read or compare job;
- Check if exception syndrome Indicates DSI (or MCI) reason;
- Data address which cause the exception matches address currently accessed by pending flash read or flash compare job.

If these conditions are verified the IVOR handler driver functions return **FLS_HANDLED_SKIP** and set the job to failed value. This information can be retrieved using **Fls_GetJobResult()** which will return **MEMIF_JOB_FAILED** or, if the job error notification parameter is configured, the notification function will be called.

With these information the following recovery strategies may be implemented:

- skip the instruction that caused the error (**FLS_HANDLED_SKIP**);
- retry the execution (**FLS_HANDLED_RETRY**);
- perform a controlled shutdown of current activity (**FLS_HANDLED_STOP**);
- do nothing (infinite loop).

IVOR handler driver functions is not available for ISI (Instruction Storage Interrupt).

In addition to this information, a basic flow is depicted in the following figure:

Implementing an Exception Handler in case of non correctable ECC error

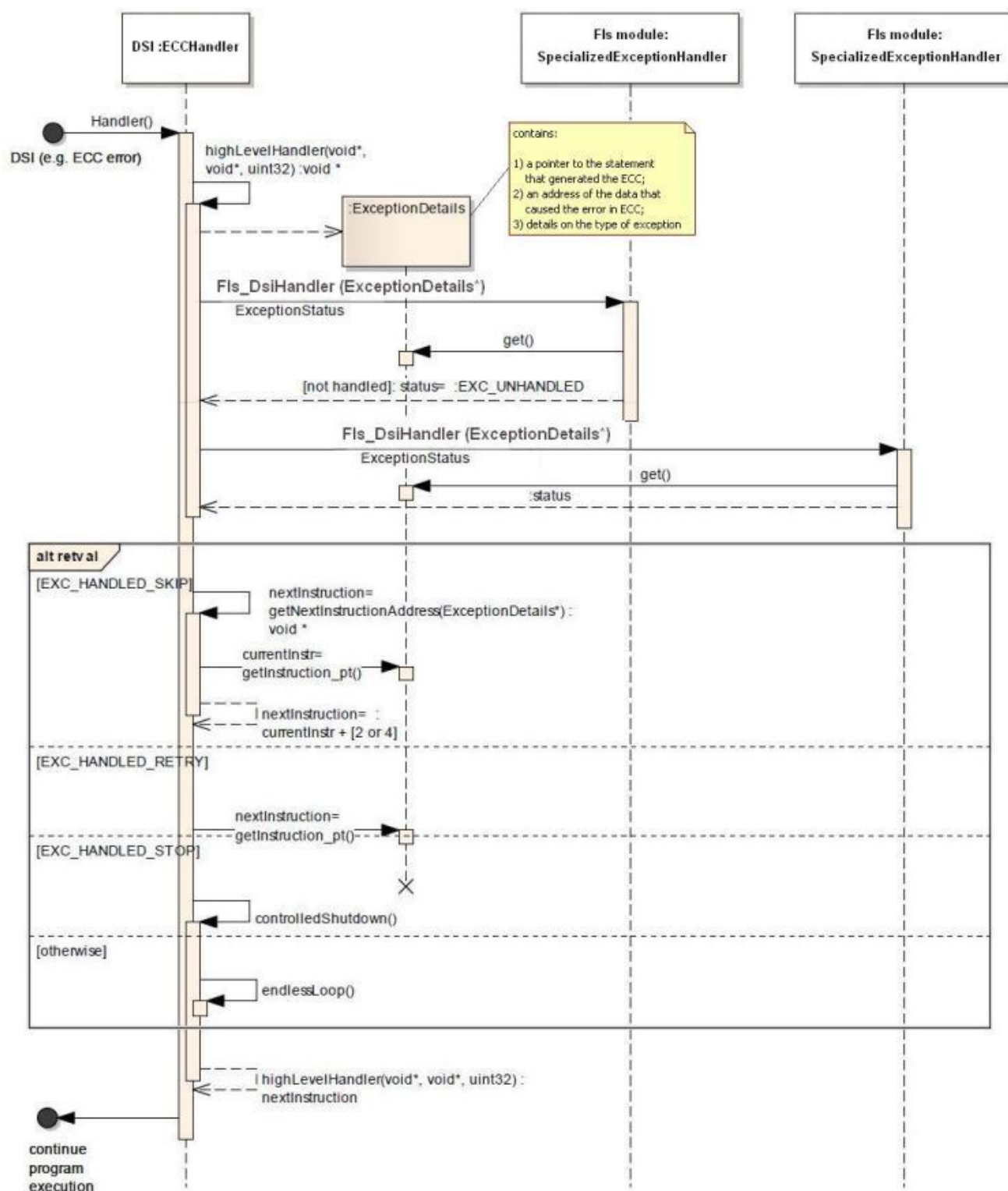


Figure 4-1. Example of ECC Handling

Chapter 5

Module requirements

None.

5.1 Exclusive areas to be defined in BSW scheduler

- **Fls_MainFunction Exclusive Areas**

Fls_Mainfunction() API contains the following critical sections:

1. FLS_EXCLUSIVE_AREA_00
2. FLS_EXCLUSIVE_AREA_01
3. FLS_EXCLUSIVE_AREA_02
4. FLS_EXCLUSIVE_AREA_03
5. FLS_EXCLUSIVE_AREA_04
6. FLS_EXCLUSIVE_AREA_05
7. FLS_EXCLUSIVE_AREA_06
8. FLS_EXCLUSIVE_AREA_07
9. FLS_EXCLUSIVE_AREA_08
10. FLS_EXCLUSIVE_AREA_10

Note: These critical sections are the result of unclear Fls SWS document requirement number FLS215.

FLS215: “The FLS module’s flash access routines shall only disable interrupts and wait for the completion of the erase/write command if necessary (that is if it has to be ensured that no other code is executed in the meantime).”

On the contrary no BSW module is allowed to directly control the global ECU interrupts, the SchM (and OS) module shall be used for this purpose. The actual implementation/behavior of this critical section is left on the ECU integrator. It means in case no other executed code (tasks) access the ‘code’ or ‘constant data’ from affected Flash area

(sectors) which is being modified by current Fls job (erase or write operations) then the implementation could be 'void' (as there is no Flash read-while-write error possible). In all other cases the execution of the code (tasks) which would access this affected Flash area (sectors) should be blocked.

Within these critical sections the Fls driver also accesses subset of the Flash PFC (Platform Flash Controller) registers (like PFCR0 or PFCR1) that can be potentially also used by Mcu driver, hence these are also protected against simultaneous access from both Fls and Mcu drivers.

Note: The exclusive areas FLS_EXCLUSIVE_AREA_00 and FLS_EXCLUSIVE_AREA_02 are valid for synchronous mode only. The following operations are executed:

1. Call Access Code Routine (sets the EHV bit in the MCR register and waits for the high-voltage operation to finish)
2. Invalidation and restore of the PFCR register.

As the duration of the last step can be longer than a single write operation, the interrupts can be disabled for an interval exceeding the erase/write timing described in the datasheet.

The invalidation and restoration of the PFCR register cannot be avoided.

Workaround for this behavior are:

1. Disable Fls_Cancel and use asynchronous mode (no critical section)
2. Implement both critical sections without long-lasting interrupts

- **Fls jobs Exclusive Area**

The exclusive area FLS_EXCLUSIVE_AREA_10 is to make the Fls_Erase, Fls_Write, Fls_Read, and Fls_Compare functions thread safe. This exclusive area protects Fls internal job variables and thus it shall use (map to) same system resource in terms of OS objects.

The exclusive areas FLS_EXCLUSIVE_AREA_03/04/05/06/07/08 are used to protect Fls_InvalidRestore function (invalidation of prefetch buffer).

5.2 Peripheral Hardware Requirements

The FLS driver uses/controls the "Flash Memory" MCU peripheral. For more details about peripheral and its structure refers to MCU reference manual.

5.3 ISR to configure within OS – dependencies

None.

5.4 ISR Macro

None.

5.5 Other AUTOSAR modules - dependencies

- **Base** This module provides basic data types and auxiliary macros or functions commonly used by other AUTOSAR modules.
- **Dem:** This module is necessary for enabling reporting of production relevant error status. The API function used is Dem_ReportErrorStatus().
- **Det** This module is necessary for enabling Development error detection. The API function used is Det_ReportError(). The activation/deactivation of Development error detection is configurable using 'CanDevErrorDetect' configuration parameter.
- **Rte:** Exclusive areas implementations.
- **MemIf:** Memory Interface
- **Resource:** Sub-Derivative model is selected from Resource configuration.
- **DSI(Data Storage Interrupt) and MCI(Machine Check Interrupt):** (only if FlsDsiHandlerApi=true)

Chapter 6

Main API Requirements

6.1 Main functions calls within Rte module

Fls_MainFunction (call rate depends on target application, i.e. how fast the data needs to be read/written/compared into Flash memory).

6.2 API Requirements

None

6.3 Calls to Notification Functions, Callbacks, Callouts

The FLS driver provides notifications that are user configurable:

- FlsAcCallback (usually routed to Wdg module)
- FlsJobEndNotification (usually routed to Fee module)
- FlsJobErrorNotification (usually routed to Fee module)

6.4 Tips for FLS integration

Synchronous vs. Asynchronous write mode

Asynchronous write mode works in the way, that Fls_MainFunction() just schedules the HW write operation and does not wait for its completion. In the next Fls_MainFunction() it is checked if the write operation is finished. If yes (depends on how often the

Fls_MainFunction() is called), another write operation is scheduled. This process is repeated until all data is written. In this mode, FlsMaxWriteFastMode/FlsMaxWriteNormalMode values are ignored, data is written just by FlsPageSize length.

When synchronous write mode is used, Fls_MainFunction() initializes write operation and also waits for its completion.

So the main differences between these two modes are in the time consumption and number of calls of the Fls_MainFunction(). The Fls_MainFunction() takes less time in asynchronous mode, but the whole write operation uses more Fls_MainFunction() executions.

Example1:

Table 6-1. Example: Synchronous vs. Asynchronous write mode

Fls_MainFunction()	Asynchronous mode	Synchronous mode
Time consumption (avrg/max)	15,7/ 96,8 μs	62,3/ 169,3 μs
Calls needed (avrg/max)	13/ 680	5/377

Example2:

FLS Max Write = 16 Byte, FLS Page Size = 8 Bytes and we are going to write 32 Bytes. In asynchronous write mode it will take at least 5 Fls_MainFunctions() (4 x 8 Bytes + 1 finish job check). In synchronous write mode it will take just 2 Fls_MainFunctions() to finish the write job (2x 16 bytes).

Possible values after interrupted HW write

Following value can be read from the flash when the HW write operation is interrupted:

1. Valid value (no ECC exception) and correct (the same as was intended to be written).
2. Valid value (no ECC exception) but incorrect (not the same as was intended to be written) – write operation was interrupted when ECC was not fully written. 1 bit error was improperly detected which lead to unwanted data correction. Example: we are going to write 00 C4 00 00, but the write operation is interrupted by reset. After reset we can see that the flash contains value 10 C4 00 00.
3. Always wrong value (stable ECC error).
4. Value with low margin – sometimes valid value is read (scenario 1 or 2) and sometimes ECC.

Note: One FLS HW write operation writes “FlsPageSize” Bytes from higher to lower addresses. It is possible to interrupt (e.g. by reset) this job after each byte.

Example: value 00 00 00 00 shall be written to the FLS. We can reset the write operation to obtain values: FF FF FF 00, FF FF 00 00 or FF 00 00 00 (of course not all of them can be seen because of ECC error. This is just an example to explain the write operation process).

Chapter 7

Memory Allocation

7.1 Sections to be defined in MemMap.h

For Post Build data:

```
#ifdef FLS_START_CONFIG_DATA_UNSPECIFIED
#undef FLS_START_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FLS_STOP_CONFIG_DATA_UNSPECIFIED
#undef FLS_STOP_CONFIG_DATA_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

For Code:

```
#ifdef FLS_START_SEC_CODE
#undef FLS_START_SEC_CODE
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FLS_STOP_SEC_CODE
#undef FLS_STOP_SEC_CODE
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

For Variables:

```
#ifdef FLS_START_SEC_VAR_UNSPECIFIED
#undef FLS_START_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FLS_STOP_SEC_VAR_UNSPECIFIED
#undef FLS_STOP_SEC_VAR_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
```

```
#endif
```

For Constant data:

```
#ifdef FLS_START_SEC_CONST_UNSPECIFIED
#undef FLS_START_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
#ifdef FLS_STOP_SEC_CONST_UNSPECIFIED
#undef FLS_STOP_SEC_CONST_UNSPECIFIED
#undef MEMMAP_ERROR
/*no definition -> default compiler settings are used */
#endif
```

For Ram Code:

```
#ifdef FLS_START_SEC_RAMCODE
#undef FLS_START_SEC_RAMCODE
#undef MEMMAP_ERROR
#pragma section CODE ".ramcode" far-absolute
#endif
#ifdef FLS_STOP_SEC_RAMCODE
#undef FLS_STOP_SEC_RAMCODE
#undef MEMMAP_ERROR
#pragma section CODE
#endif
```

7.2 Linker command file

- **Section definition**

Memory shall be allocated for every section defined in MemMap.h.

- **Access code loaded at job start**

Additionally if **FlsAcLoadOnJobStart=true** some definitions should be put in the linker file.

These definitions are used by FLS driver to copy the access code from ROM to RAM

1. FlsACEraseStart and FlsACWriteStart point to the beginning of ROM section where erase/write access code has been placed
2. FlsACEraseEnd and FlsACWriteEnd point to the ending of ROM section where erase/write access code has been placed
3. FlsACSizeErase and FlsACSizeWrite are the respective sizes [in bytes] of the erase access code and the write access code sections

Example with GHS linker file

In the following example, write and erase access codes are identical

res_ram has been declared in the RAM memory mapping

The memory section **".acfls_code"** is used with FLS_START_SEC_CODE_AC directive

```
// RAM section
...
.acfls_code    ALIGN(4) : > res_ram
...
// ROM section
...
.ROM.acfls_code ROM(.acfls_code) ALIGN(0x10) : > .
...
FLS_AC_SRC = ADDR(.ROM.acfls_code);
FLS_AC_SIZE = (SIZEOF(.ROM.acfls_code));

Fls_AC_EraseStart      = FLS_AC_SRC;
Fls_AC_EraseEnd        = FLS_AC_SRC + FLS_AC_SIZE;
Fls_AC_EraseSize       = (FLS_AC_SIZE) / 4;

Fls_AC_WriteStart      = FLS_AC_SRC;
Fls_AC_WriteEnd        = FLS_AC_SRC + FLS_AC_SIZE;
Fls_AC_WriteSize       = (FLS_AC_SIZE) / 4;
```

When using following configuration parameters

1. FlsAcErasePointer
2. FlsAcWritePointer

It is possible to use symbolic name instead of absolute addresses, but in this case the linker should define them

Example with GHS linker file with _ERASE_FUNC_ADDRESS_ and _WRITE_FUNC_ADDRESS_ symbolic names

```
FLS_AC_DEST = ADDR(.acfls_code);
_ERASE_FUNC_ADDRESS_ = FLS_AC_DEST;
_WRITE_FUNC_ADDRESS_ = FLS_AC_DEST;
```

• Invalid Restore FLS function

Function Fls_InvalidRestore_Ram should be executed from RAM, FLS_START_SEC_RAMCODE and FLS_STOP_SEC_RAMCODE have been implemented for this purpose in MemMap.h which defines a new pragma section **".ramcode"**.

In the linker command file **".ramcode"** section should be defined; then a separate routine should be implemented by the user to copy this section from flash to ram, for example into the startup file.

Chapter 8

Configuration parameters considerations

Configuration parameter class for Autosar Fls driver fall into the following variants as defined below:

8.1 Configuration Parameters

Specifies whether the configuration parameter shall be of configuration class Post Build or PreCompile.

Table 8-1. Configuration Parameters

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
FlsGeneral			
	FlsAcLoadOnJobStart	VariantPostBuild	PreCompile
	FlsDisableDemReportErrorStatus	VariantPostBuild	PreCompile
	FlsBaseAddress	VariantPostBuild	PreCompile
	FlsCancelApi	VariantPostBuild	PreCompile
	FlsCompareApi	VariantPostBuild	PreCompile
	FlsDevErrorDetect	VariantPostBuild	PreCompile
	FlsDriverIndex	VariantPostBuild	PreCompile
	FlsGetJobResultApi	VariantPostBuild	PreCompile
	FlsGetStatusApi	VariantPostBuild	PreCompile
	FlsSetModeApi	VariantPostBuild	PreCompile
	FlsTotalSize	VariantPostBuild	PreCompile
	FlsUseInterrupts	VariantPostBuild	PreCompile
	FlsVersionInfoApi	VariantPostBuild	PreCompile
	FlsDsiHandlerApi	VariantPostBuild	PreCompile
	FlsDsiHandlerInclude	VariantPostBuild	PreCompile
	FlsEraseBlankCheck	VariantPostBuild	PreCompile
	FlsWriteBlankCheck	VariantPostBuild	PreCompile
	FlsWriteVerifyCheck	VariantPostBuild	PreCompile

Table continues on the next page...

Table 8-1. Configuration Parameters (continued)

Configuration Container	Configuration Parameters	Configuration Variant	Current Implementation
FlsConfigSet	FlsMaxEraseBlankCheck	VariantPostBuild	PreCompile
	FlsAcErase	VariantPostBuild	PostBuild
	FlsAcWrite	VariantPostBuild	PostBuild
	FlsAcErasePointer	VariantPostBuild	PostBuild
	FlsAcWritePointer	VariantPostBuild	PostBuild
	FlsCallCycle	VariantPostBuild	PostBuild
	FlsAcCallback	VariantPostBuild	PostBuild
	FlsJobEndNotification	VariantPostBuild	PostBuild
	FlsJobErrorNotification	VariantPostBuild	PostBuild
	FlsMaxReadFastMode	VariantPostBuild	PostBuild
	FlsMaxReadNormalMode	VariantPostBuild	PostBuild
	FlsMaxWriteFastMode	VariantPostBuild	PostBuild
	FlsMaxWriteNormalMode	VariantPostBuild	PostBuild
	FlsProtection	VariantPostBuild	PostBuild
FlsSector			
	FlsPhysicalSectorUnlock	VariantPostBuild	PostBuild
	FlsPhysicalSector	VariantPostBuild	PostBuild
	FlsNumberOfSectors	VariantPostBuild	PostBuild
	FlsPageSize	VariantPostBuild	PostBuild
	FlsSectorSize	VariantPostBuild	PostBuild
	FlsSectorStartaddress	VariantPostBuild	PostBuild
	FlsProgrammingSize	VariantPostBuild	PostBuild
	FlsSectorEraseAsynch	VariantPostBuild	PostBuild
	FlsPageWriteAsynch	VariantPostBuild	PostBuild

Chapter 9

Integration Steps

This section gives a brief overview of the steps needed for integrating Flash :

- Generate the required Fls configurations. For more details refer to section [Files required for Compilation](#)
- Allocate proper memory sections in MemMap.h and linker command file. For more details refer to section [Sections to be defined in MemMap.h](#)
- Compile & build the Fls with all the dependent modules. For more details refer to section [Building the Driver](#)





Chapter 10

ISR Reference

ISR functions exported by the Fls driver.

10.1 Software specification

The following sections contains driver software specifications.

10.1.1 Define Reference

Constants supported by the Fls driver.

10.1.2 Macros Reference

Macros supported by the Fls driver.

10.1.3 Enum Reference

Enumeration of all constants supported by the Fls driver.

10.1.4 Function Reference

Functions supported by the Fls driver.

10.1.5 Structs Reference

Data structures supported by the Fls driver.

10.1.6 Types Reference

Types structures supported by the Fls driver.

10.1.7 Variables Reference

Variables supported by the Fls driver.

How to Reach Us:**Home Page:**freescale.com**Web Support:**freescale.com/support

Information in this document is provided solely to enable system and software implementers to use Freescale products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document.

Freescale reserves the right to make changes without further notice to any products herein. Freescale makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does Freescale assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in Freescale data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. Freescale does not convey any license under its patent rights nor the rights of others. Freescale sells products pursuant to standard terms and conditions of sale, which can be found at the following address: freescale.com/SalesTermsandConditions.

Freescale, the Freescale logo, Altivec, C-5, CodeTest, CodeWarrior, ColdFire, ColdFire+, C-Ware, Energy Efficient Solutions logo, Kinetis, mobileGT, PowerQUICC, Processor Expert, QorIQ, Qorivva, StarCore, Symphony, and VortiQa are trademarks of Freescale Semiconductor, Inc., Reg. U.S. Pat. & Tm. Off. Airfast, BeeKit, BeeStack, CoreNet, Flexis, Layerscape, MagniV, MXC, Platform in a Package, QorIQ Qonverge, QUICC Engine, Ready Play, SafeAssure, SafeAssure logo, SMARTMOS, Tower, TurboLink, Vybrid, and Xtrinsic are trademarks of Freescale Semiconductor, Inc. All other product or service names are the property of their respective owners.

© 2014 Freescale Semiconductor, Inc.

Document Number IM13FLSASR4.0 Rev0003R1.0.1
Revision 3.0